



GUÍA DE ARQUITECTURA Y BUENAS PRÁCTICAS DE DESARROLLO

Este documento técnico, dirigido a la Gobernación de Antioquia, establece el marco de referencia para el desarrollo de software en la entidad pública. Integra estándares internacionales y nacionales como MinTIC, MRAE, ISO 27001 y OWASP, y está orientado a garantizar la sostenibilidad, seguridad y calidad de los activos de información. Su propósito principal es guiar a los equipos tecnológicos —internos y externos— en la aplicación de buenas prácticas, promoviendo la interoperabilidad y el valor público del software desarrollado. El alcance incluye todos los sistemas y aplicativos cuyo derecho patrimonial pertenece a la Gobernación, involucrando a funcionarios, contratistas y proveedores en su implementación.

Contenido

1. Introducción y Contexto Estratégico	2
1.1. Introducción	2
1.2. Propósito	2
1.3. Objetivo	2
1.4. Alcance y Propiedad Intelectual	2
2. Principios Rectores	2
3. Arquitectura de Referencia	3
3.1. Estilos de Arquitectura	3
3.2. Patrones de Diseño Interno	3
3.3. Integración	3
4. Stack Tecnológico de Referencia	3
5. Buenas Prácticas de Desarrollo	4
6. Estrategia de Entornos y Gestión de Datos	4
6.1. Definición de Entornos	4
6.2. Gestión de Datos de Prueba (Test Data Management)	5
7. Ciclo de Desarrollo y Entregables Sugeridos	5
8. Pruebas y Calidad (QA)	5
9. Seguridad y Privacidad (Obligatorio)	6
10. Prácticas de Documentación	6

1. Introducción y Contexto Estratégico

1.1. Introducción

El presente documento constituye el **marco de referencia técnico** para el ciclo de vida de desarrollo de software dentro de la Entidad. Esta guía integra las mejores prácticas de la industria con las exigencias normativas del Estado Colombiano, sirviendo como brújula técnica para los equipos de tecnología, independientemente de si son internos (funcionarios) o externos (contratistas y proveedores).

1.2. Propósito

El propósito fundamental es orientar a los equipos hacia la sostenibilidad, seguridad, mantenibilidad y calidad de los activos de información. Se busca facilitar la toma de decisiones técnicas que mitiguen la deuda técnica y la obsolescencia, promoviendo que el software aporte valor público y cumpla con los estándares de calidad esperados.

1.3. Objetivo

Proveer un catálogo unificado de patrones de arquitectura, tecnologías de referencia y buenas prácticas de codificación. Este instrumento actúa como facilitador para mejorar la interoperabilidad y la calidad del software, optimizando los recursos públicos invertidos en tecnología.

1.4. Alcance y Propiedad Intelectual

Esta guía es de aplicación para todos los desarrollos de software, sistemas de información y aplicativos móviles cuya **titularidad patrimonial y derechos de propiedad intelectual pertenezcan a la Gobernación de Antioquia**.

- **Actores:** Funcionarios, contratistas, fábricas de software y proveedores.
- **Proyectos:** Nuevos desarrollos y mantenimientos evolutivos.
- **Derechos de Autor:** Todo código fuente generado bajo contrato o relación laboral es propiedad de la entidad. Se debe garantizar la entrega total de fuentes, scripts y documentación sin ofuscación.

2. Principios Rectores

Fundamentos que orientan las decisiones técnicas.

- **Enfoque Ciudadano (Digital First):** Se debe priorizar que todo servicio digital sea accesible y usable para el ciudadano (Res. 1519 de 2020).
- **Seguridad y Privacidad por Diseño (PbD):** La seguridad es un requisito funcional desde el inicio. **Se debe** cumplir estrictamente la Ley 1581 (Habeas Data).

- **Neutralidad Tecnológica y Preferencia Open Source:** Se sugiere priorizar el uso de software de código abierto (OSS) para eficiencia del gasto, salvo justificación técnica o de negocio contraria.
- **Capacidad de Interoperabilidad:** Se recomienda diseñar arquitecturas desacopladas que faciliten, cuando sea requerido, la integración mediante servicios estandarizados (X-Road / Servicios Ciudadanos Digitales).
- **Calidad Sostenible:** Se promueve la gestión proactiva de la deuda técnica para mantener el código legible y mantenible.

3. Arquitectura de Referencia

Modelos sugeridos según la complejidad del problema de negocio.

3.1. Estilos de Arquitectura

- **Monolito Modular:** Recomendado para aplicaciones donde el dominio es unificado y se busca simplicidad operativa.
 - *Características:* Un solo artefacto de despliegue, con modularización lógica interna (*packages* o *namespaces*).
- **Arquitectura Orientada a Servicios (SOA):** Recomendada para escenarios de integración donde múltiples sistemas comparten lógica de negocio o datos.
 - *Características:* Servicios de grano grueso y contratos estandarizados.
- **Microservicios:** Recomendado solo para sistemas que requieren escalabilidad granular extrema o despliegues independientes frecuentes.
 - *Características:* Servicios de grano fino, base de datos por servicio; requiere alta madurez en orquestación y observabilidad.

3.2. Patrones de Diseño Interno

- **Clean Architecture / Arquitectura Hexagonal:** Patrón altamente recomendado para el backend.
 - *Principio:* Separar el Dominio (Núcleo) de la Infraestructura (Frameworks/BD) para proteger las reglas de negocio de la obsolescencia tecnológica.

3.3. Integración

- **API First:** Se sugiere diseñar los contratos (OpenAPI/Swagger) antes de iniciar la codificación.
- **Servicios Ciudadanos Digitales (SCD):** Se recomienda evaluar la integración con Autenticación Digital y Carpeta Ciudadana cuando el usuario final sea el ciudadano.

4. Stack Tecnológico de Referencia

Catálogo de tecnologías soportadas y sugeridas. La elección final debe basarse en la idoneidad técnica para el problema específico.

Capa	Tecnologías de Referencia
Frontend	Angular, React, Vue.js, Svelte, TypeScript, JavaScript (Vanilla).
Backend	Java (Spring Boot), Python (FastAPI/Django), C# (.NET), Node.js, PHP, Go.
Base de Datos	PostgreSQL, MySQL/MariaDB, Oracle, SQL Server, MongoDB, DynamoDB.
Caché/Bus	Redis, RabbitMQ, Kafka, ActiveMQ, Memcached.
Móvil	Flutter, React Native, PWA, Desarrollo Nativo (Swift/Kotlin).

5. Buenas Prácticas de Desarrollo

Recomendaciones tácticas para la escritura de código.

- **Estrategia de Ramas:** Se sugiere el uso de **GitFlow** o modelos similares que ordenen el trabajo colaborativo.
 - main: Producción estable.
 - develop: Integración continua.
- **Commits:** Se recomienda adoptar el estándar "Conventional Commits" (ej. feat:, fix:) para mantener un historial legible.
- **Idioma:** Se establece el uso del Español (neutro) en código y comentarios para garantizar la consistencia y entendimiento transversal en la entidad.
- **Gestión del Ciclo de Vida:** Se recomienda seguir los lineamientos operativos de la **Guía de Uso de Azure DevOps Institucional** para la configuración eficiente de repositorios y tableros.

6. Estrategia de Entornos y Gestión de Datos

Segregación de ambientes para garantizar la estabilidad y seguridad.

6.1. Definición de Entornos

Se recomienda disponer de al menos tres ambientes lógicos o físicos separados:

1. **Desarrollo (Dev):** Entorno volátil para uso de los desarrolladores. Integración continua constante.

2. **Calidad / Pruebas (QA/Staging):** Réplica lo más fiel posible a producción en infraestructura. Uso exclusivo para certificación funcional, pruebas de carga y seguridad.
3. **Producción (Prod):** Entorno final accesible por el usuario. **Solo se despliega aquí tras aprobación de QA.**

6.2. Gestión de Datos de Prueba (Test Data Management)

- **Anonimización Obligatoria:** Está **prohibido** el uso de datos reales de producción (ciudadanos reales, historias clínicas, datos financieros) en los entornos de Desarrollo y QA sin un proceso previo de enmascaramiento o anonimización.
- **Datos Sintéticos:** Se recomienda el uso de generadores de datos ficticios (Mock Data) para las pruebas unitarias y de integración temprana.
- **Limpieza:** Los entornos no productivos deben tener políticas de limpieza periódica de datos.

7. Ciclo de Desarrollo y Entregables Sugeridos

Alineación con las fases de valor de MIPG.

1. Fase de Diseño:

- *Entregable:* Documento de Arquitectura de Software (DAS) o diagramas de contexto (C4).
- *Entregable:* Prototipos de UI/UX.

2. Fase de Construcción (Sprints):

- Ejecución de Pruebas Unitarias.
- Ejecución de Análisis Estático (Linters).

3. Fase de Transición:

- *Entregable:* Manual de Despliegue e Instalación.
- *Entregable:* Resultado de escaneo de vulnerabilidades.
- *Entregable:* Código fuente completo en repositorio institucional.

8. Pruebas y Calidad (QA)

Estrategia de aseguramiento de calidad.

- **Cobertura:** Se sugiere mantener una cobertura de pruebas unitarias superior al 80% en la lógica de negocio crítica.
- **Tipos de Pruebas Recomendadas:**
 - *Unitarias:* (JUnit, Jest).
 - *Integración:* (Postman/Newman).
 - *Seguridad (SAST):* Análisis de dependencias.
 - *Accesibilidad:* Validación WCAG 2.1 AA (WAVE).

- **Generación de Artefactos de Auditoría:**

- Reportes de cobertura y ejecución de pruebas generados automáticamente en el pipeline.
- Logs de análisis estático exitoso.

9. Seguridad y Privacidad (Obligatorio)

Cumplimiento normativo y protección de activos.

- **Gestión de Identidad:**

- **Prohibido** almacenar contraseñas en texto plano.
- Implementación de MFA para administradores, sugiriendo integración con Microsoft Entra ID (Azure AD).

- **Protección de Datos:**

- **Se debe** cifrar la base de datos en reposo (TDE) cuando contenga datos sensibles.
- **Obligatorio** el cifrado en tránsito (TLS 1.2+).
- **Ley 1581:** La Política de Privacidad debe ser visible y aceptada explícitamente.

- **Licenciamiento y Propiedad Intelectual:**

- Se debe verificar que las librerías de terceros (Open Source) tengan licencias compatibles con el uso institucional (ej. MIT, Apache 2.0).
- Se debe evitar el uso de librerías con licencias restrictivas (ej. GPL) si estas obligan a liberar el código propietario de la Gobernación, salvo autorización jurídica.

- **Normativa Institucional:** **Se debe** dar cumplimiento estricto a las políticas de seguridad de la información definidas por la entidad.

10. Prácticas de Documentación

Gestión del conocimiento.

- **Técnica:** Se recomienda incluir un archivo README.md detallando prerequisites, instalación y variables de entorno.
- **Funcional:** Registro de Historias de Usuario con Criterios de Aceptación claros.
- **API:** Documentación de servicios mediante estándares como Swagger/OpenAPI.